

ASSIGNMENT 2

DATA ANALYTICS USING PYTHON

NAME: MAITREYINI S

COURSE/INTERNSHIP NAME: DATA ANALYTICS INTERNSHIP

TASK 1.1 – PYTHON BASICS

Python Code

```
i = 25

f = 3.14

s = "Data Analytics"

b = True

lst = [10, 20, 30]

tup = (1, 2, 3)

dct = {"name": "Aarav", "age": 28}

print("===== TASK 1.1 - DATA TYPES =====")

print("INTEGER:", i, type(i))

print("FLOAT:", f, type(f))

print("STRING:", s, type(s))

print("BOOLEAN:", b, type(b))

print("LIST:", lst, type(lst))

print("TUPLE:", tup, type(tup))

print("DICTIONARY:", dct, type(dct))

a = 10

b_num = 3

print("\n===== ARITHMETIC OPERATORS =====")

print("Addition:", a + b_num)
```

```
print("Subtraction:", a - b_num)
print("Multiplication:", a * b_num)
print("Division:", a / b_num)
print("Modulus:", a % b_num)
print("Power:", a ** b_num)
print("Floor Division:", a // b_num)
print("\n===== COMPARISON OPERATORS =====")
print("Equal:", a == b_num)
print("Greater than:", a > b_num)
print("Less than:", a < b_num)
print("\n===== LOGICAL OPERATORS =====")
print("AND:", a > 5 and b_num < 5)
print("OR:", a < 5 or b_num < 5)
```

Output Screenshot

```
>>> # Arithmetic Operators
>>>
>>> a = 10
>>> b_num = 3
>>>
>>> print("Addition:", a + b_num)
Addition: 13
>>> print("Subtraction:", a - b_num)
Subtraction: 7
>>> print("Multiplication:", a * b_num)
Multiplication: 30
>>> print("Division:", a / b_num)
Division: 3.3333333333333335
>>> print("Modulus:", a % b_num)
Modulus: 1
>>> print("Power:", a ** b_num)
Power: 1000
>>> print("Floor Division:", a // b_num)
Floor Division: 3
>>>
>>> # Comparison Operators
>>>
>>> print("Equal:", a == b_num)
Equal: False
>>> print("Greater than:", a > b_num)
Greater than: True
>>> print("Less than:", a < b_num)
Less than: False
>>>
>>> # Logical Operators
>>>
>>> print("AND:", a > 5 and b_num < 5)
AND: True
>>> print("OR:", a < 5 or b_num < 5)
OR: True
```

TASK 1.2 – NUMPY OPERATIONS

Python Code

```
import numpy as np

sales = np.array([125, 138, 142, 155, 168, 180, 195, 210, 225, 240, 258, 275])

print("\n===== TASK 1.2 - NUMPY OPERATIONS =====")

print("Shape:", sales.shape)

print("Size:", sales.size)

print("Dtype:", sales.dtype)

print("Sum:", sales.sum())

print("Mean:", sales.mean())

print("Maximum:", sales.max())

print("Minimum:", sales.min())

print("Standard Deviation:", sales.std())

reshaped = sales.reshape(4, 3)

print("\nReshaped 4 x 3:")

print(reshaped)

high_sales_idx = np.where(sales > 200)

print("\nIndices where sales are greater than 200:")

print(high_sales_idx)

growth = np.diff(sales)

print("\nMonth-over-month growth:")

print(growth)
```

Output Screenshot

```
>>> print("Reshaped 4 x 3:")
Reshaped 4 x 3:
>>> print(reshaped)
[[125 138 142]
 [155 168 180]
 [195 210 225]
 [240 258 275]]
>>>
>>> # Indices where sales are greater than 200
>>> high_sales_idx = np.where(sales > 200)
>>>
>>> print("Indices where sales > 200:")
Indices where sales > 200:
>>> print(high_sales_idx)
(array([ 7,  8,  9, 10, 11]),)
>>>
>>> # Month-over-month growth
>>> growth = np.diff(sales)
>>>
>>> print("Month-over-month growth:")
Month-over-month growth:
>>> print(growth)
[13  4 13 13 12 15 15 15 15 18 17]
```

TASK 2.1 – PANDAS: LOADING AND INSPECTION

Python Code

```
import pandas as pd

import numpy as np

customers_data = {

    "cust_id": ["C01","C02","C03","C04","C05","C06","C07","C08"],

    "name": ["Aarav","Bhavya","Chirag","Diya","Eshan","Farah","Gaurav","Hina"],

    "city": ["Mumbai","Delhi","Bangalore","Mumbai","Pune","Delhi","Bangalore","Mumbai"],

    "age": [28,32,26,np.nan,30,29,40,27],

    "gender": ["M","F","M","F","M","F","M",np.nan]

}

customers_df = pd.DataFrame(customers_data)

purchases_data = {

    "purchase_id": ["P101","P102","P103","P104","P105","P106","P107","P108","P109","P110"],

    "cust_id": ["C01","C02","C03","C01","C05","C02","C06","C07","C03","C05"],

    "product": ["Laptop","Saree","Smartphone","Tablet","Headphones",

                "Watch","T-shirt","Backpack","Mouse","Charger"],

    "amount": [55000,4500,32000,18000,1500,3500,499,1800,899,650],

    "purchase_date": ["2025-02-03","2025-02-05","2025-02-09",

                      "2025-02-11","2025-02-12","2025-02-14",

                      "2025-02-18","2025-02-20","2025-02-22","2025-02-26"]

}

purchases_df = pd.DataFrame(purchases_data)

print("\n===== TASK 2.1 - PANDAS DATA =====")

print("\nCUSTOMERS - FIRST 5 ROWS:")

print(customers_df.head())

print("\nPURCHASES - FIRST 5 ROWS:")
```

```

print(purchases_df.head())

print("\nCustomers Shape:", customers_df.shape)

print("Purchases Shape:", purchases_df.shape)

print("\nCustomers Information:")

customers_df.info()

print("\nPurchases Information:")

purchases_df.info()

```

Output Screenshot

```

CUSTOMERS:
>>> print(customers_df.head())
   cust_id  name    city  age  gender
0     C01  Aarav   Mumbai  28.0     M
1     C02  Bhavya   Delhi  32.0     F
2     C03  Chirag  Bangalore  26.0     M
3     C04   Diya   Mumbai   NaN     F
4     C05  Eshan    Pune  30.0     M
>>>
>>> print("\nPURCHASES:")

PURCHASES:
>>> print(purchases_df.head())
   purchase_id  cust_id  product  amount  purchase_date
0          P101     C01   Laptop   55000   2025-02-03
1          P102     C02    Saree    4500   2025-02-05
2          P103     C03  Smartphone  32000   2025-02-09
3          P104     C01   Tablet   18000   2025-02-11
4          P105     C05  Headphones   1500   2025-02-12
>>>
>>> # Shape and information
>>> print("\nCustomers Shape:", customers_df.shape)

Customers Shape: (8, 5)
>>> print("Purchases Shape:", purchases_df.shape)
Purchases Shape: (10, 5)
>>>
>>> print("\nCustomers Information:")

Customers Information:
>>> customers_df.info()
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8 entries, 0 to 7

```

TASK 2.2 – DATA CLEANING

Python Code

```
print("\n===== TASK 2.2 - DATA CLEANING =====")

print("Missing Values:")

print(customers_df.isnull().sum())

customers_df["age"] = customers_df["age"].fillna(

    customers_df["age"].mean()

)

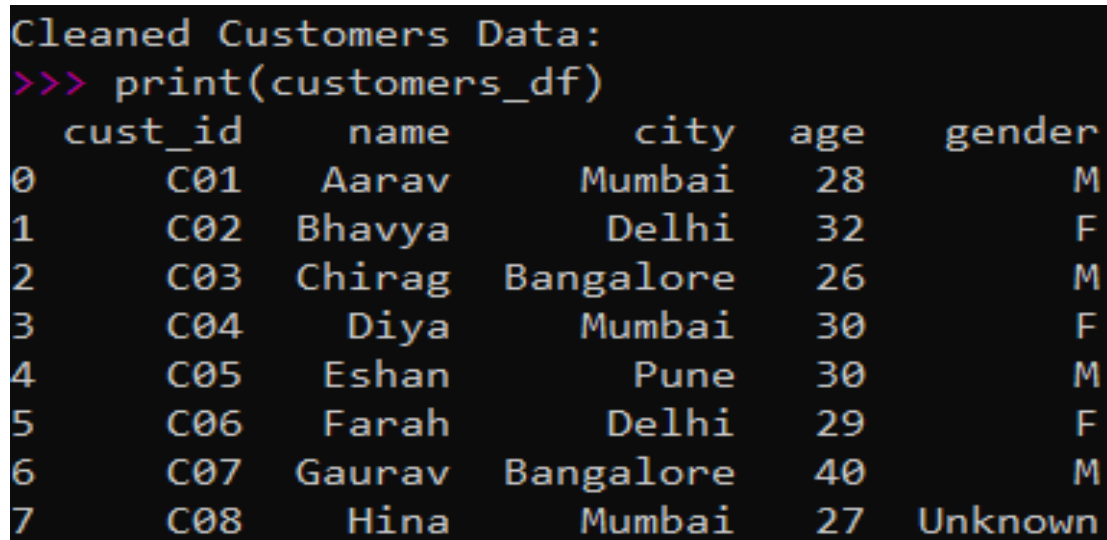
customers_df["gender"] = customers_df["gender"].fillna("Unknown")

customers_df["age"] = customers_df["age"].astype(int)

print("\nCleaned Customers Data:")

print(customers_df)
```

Output Screenshot



```
Cleaned Customers Data:
>>> print(customers_df)
   cust_id  name    city  age  gender
0     C01  Aarav  Mumbai   28      M
1     C02  Bhavya   Delhi   32      F
2     C03  Chirag  Bangalore  26      M
3     C04   Diya   Mumbai   30      F
4     C05  Eshan   Pune    30      M
5     C06  Farah   Delhi   29      F
6     C07  Gaurav  Bangalore  40      M
7     C08   Hina   Mumbai   27  Unknown
```

TASK 2.3 – DATA MERGING

Python Code

```
print("\n===== TASK 2.3 - DATA MERGING =====")

merged_df = pd.merge(
    customers_df,
    purchases_df,
    on="cust_id",
    how="inner"
)

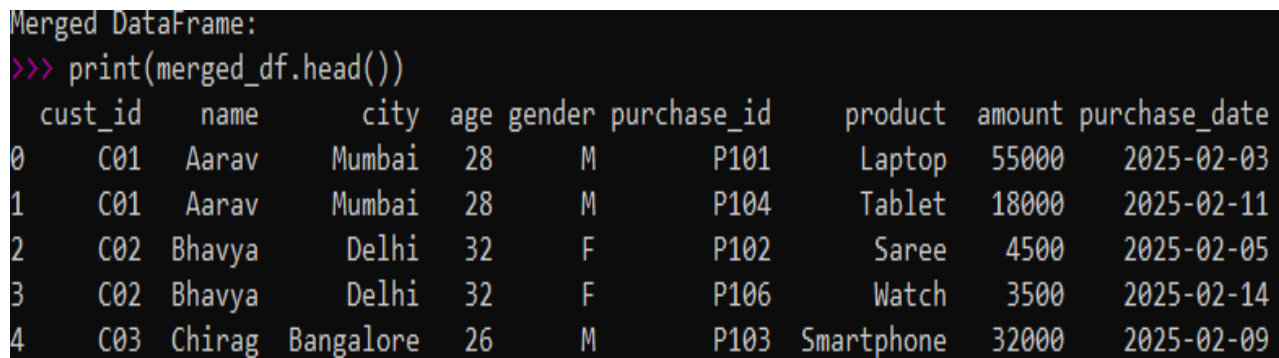
print("Merged DataFrame:")

print(merged_df.head())

print("\nMerged Shape:")

print(merged_df.shape)
```

Output Screenshot



```
Merged DataFrame:
>>> print(merged_df.head())
   cust_id  name  city  age  gender  purchase_id  product  amount  purchase_date
0     C01  Aarav  Mumbai   28     M        P101   Laptop   55000   2025-02-03
1     C01  Aarav  Mumbai   28     M        P104   Tablet   18000   2025-02-11
2     C02  Bhavya   Delhi   32     F        P102    Saree    4500   2025-02-05
3     C02  Bhavya   Delhi   32     F        P106    Watch    3500   2025-02-14
4     C03  Chirag  Bangalore  26     M        P103  Smartphone  32000   2025-02-09
```

TASK 3.1 – FILTERING

Required filters:

- All purchases from customers in Mumbai
- All purchases with amount greater than ₹5,000
- Male customers who made a purchase

Python Code

```
print("\n===== TASK 3.1 - FILTERING =====")

print("\nPurchases from Mumbai:")

mumbai_purchases = merged_df[merged_df["city"] == "Mumbai"]

print(mumbai_purchases)

print("\nPurchases above 5000:")

high_value_purchases = merged_df[merged_df["amount"] > 5000]

print(high_value_purchases)

print("\nMale customers who made purchases:")

male_purchases = merged_df[merged_df["gender"] == "M"]

print(male_purchases)
```

Output Screenshot

```
Purchases from Mumbai:
>>> print(merged_df[merged_df["city"] == "Mumbai"])
  cust_id  name    city  age gender purchase_id product  amount purchase_date
0     C01  Aarav  Mumbai   28     M        P101  Laptop   55000   2025-02-03
1     C01  Aarav  Mumbai   28     M        P104  Tablet   18000   2025-02-11
>>>
>>>
>>> # 2. All purchases with amount greater than 5000
>>> print("\nPurchases above 5000:")

Purchases above 5000:
>>> print(merged_df[merged_df["amount"] > 5000])
  cust_id  name    city  age gender purchase_id product  amount purchase_date
0     C01  Aarav  Mumbai   28     M        P101  Laptop   55000   2025-02-03
1     C01  Aarav  Mumbai   28     M        P104  Tablet   18000   2025-02-11
4     C03  Chirag  Bangalore  26     M        P103  Smartphone  32000   2025-02-09
>>>
>>>
>>> # 3. Male customers who made a purchase
>>> print("\nMale customers who made purchases:")

Male customers who made purchases:
>>> print(merged_df[merged_df["gender"] == "M"])
  cust_id  name    city  age gender purchase_id product  amount purchase_date
0     C01  Aarav  Mumbai   28     M        P101  Laptop   55000   2025-02-03
1     C01  Aarav  Mumbai   28     M        P104  Tablet   18000   2025-02-11
4     C03  Chirag  Bangalore  26     M        P103  Smartphone  32000   2025-02-09
5     C03  Chirag  Bangalore  26     M        P109    Mouse     899   2025-02-22
6     C05  Eshan    Pune   30     M        P105  Headphones   1500   2025-02-12
7     C05  Eshan    Pune   30     M        P110   Charger     650   2025-02-26
9     C07  Gaurav  Bangalore  40     M        P108  Backpack   1800   2025-02-20
```

TASK 3.2 – GROUPING & AGGREGATION

Required analyses:

- Total purchase amount per city
- Average purchase amount per gender
- Number of purchases per customer
- Top 3 customers by total spending

Python Code

```
print("\n===== TASK 3.2 - GROUPING AND AGGREGATION =====")

print("\nTotal Purchase Amount per City:")

city_totals = merged_df.groupby("city")["amount"].sum()

print(city_totals)

print("\nAverage Purchase Amount per Gender:")

gender_average = merged_df.groupby("gender")["amount"].mean()

print(gender_average)

print("\nNumber of Purchases per Customer:")

purchase_count = merged_df.groupby("cust_id").size()

print(purchase_count)

print("\nTop 3 Customers by Total Spending:")

top3 = (

    merged_df.groupby("name")["amount"]

    .sum()

    .sort_values(ascending=False)

    .head(3)

)

print(top3)
```

Output Screenshot

```
>>> # 1. Total purchase amount per city
>>> print("Total Purchase Amount per City:")
Total Purchase Amount per City:
>>> city_totals = merged_df.groupby("city")["amount"].sum()
>>> print(city_totals)
city
Bangalore    34699
Delhi         8499
Mumbai       73000
Pune         2150
Name: amount, dtype: int64
```

```
Average Purchase Amount per Gender:
>>> gender_average = merged_df.groupby("gender")["amount"].mean()
>>> print(gender_average)
gender
F      2833.000000
M     15692.714286
Name: amount, dtype: float64
```

```
Number of Purchases per Customer:
>>> purchase_count = merged_df.groupby("cust_id").size()
>>> print(purchase_count)
cust_id
C01      2
C02      2
C03      2
C05      2
C06      1
C07      1
```

```
>>> print(top3)
name
Aarav      73000
Chirag     32899
Bhavya      8000
```

TASK 3.3 – VISUALIZATIONS

FIGURE 1

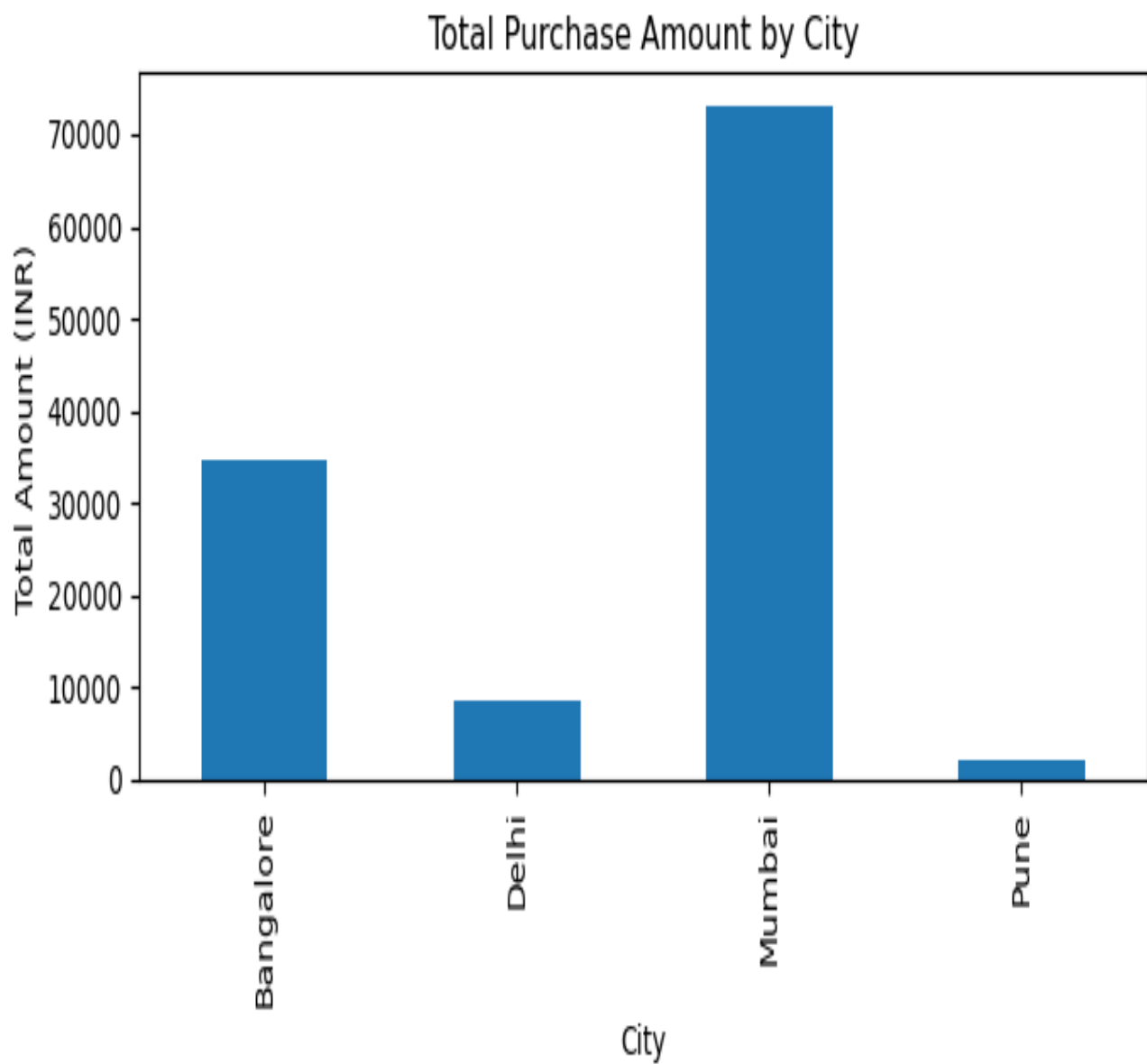


FIGURE 2

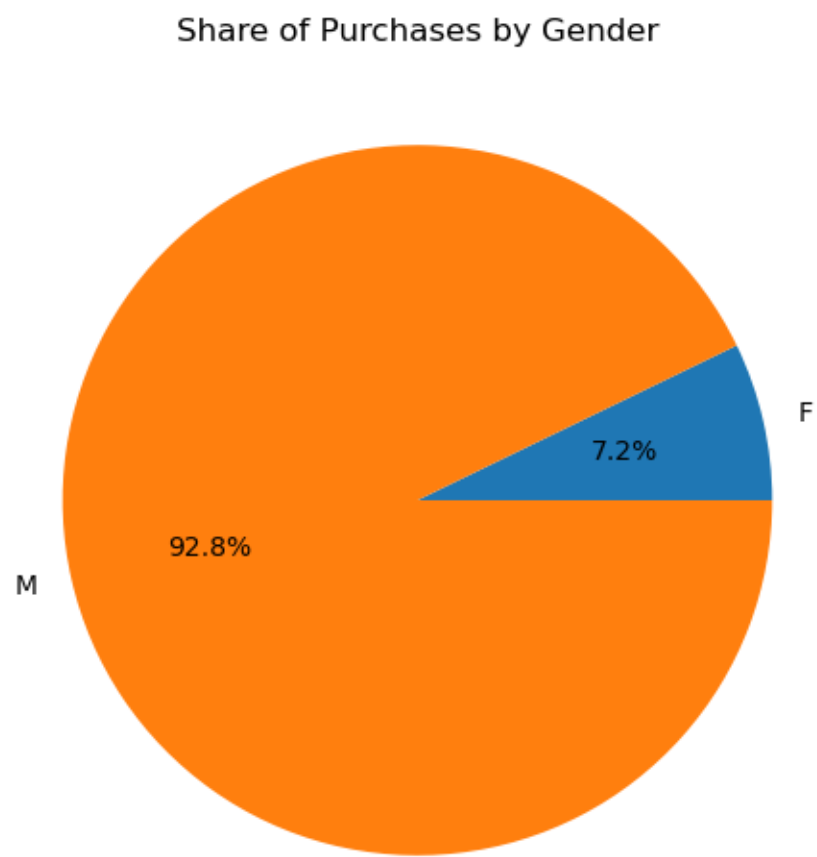


FIGURE 3

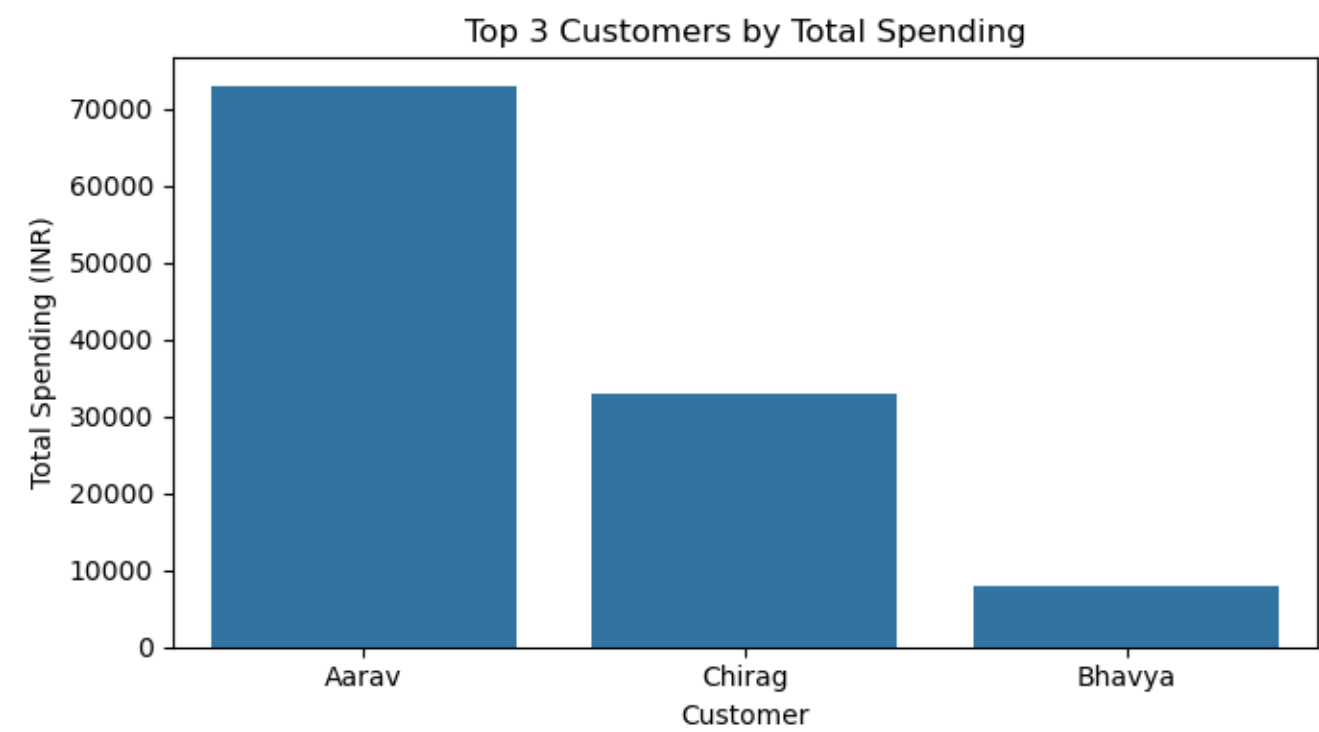
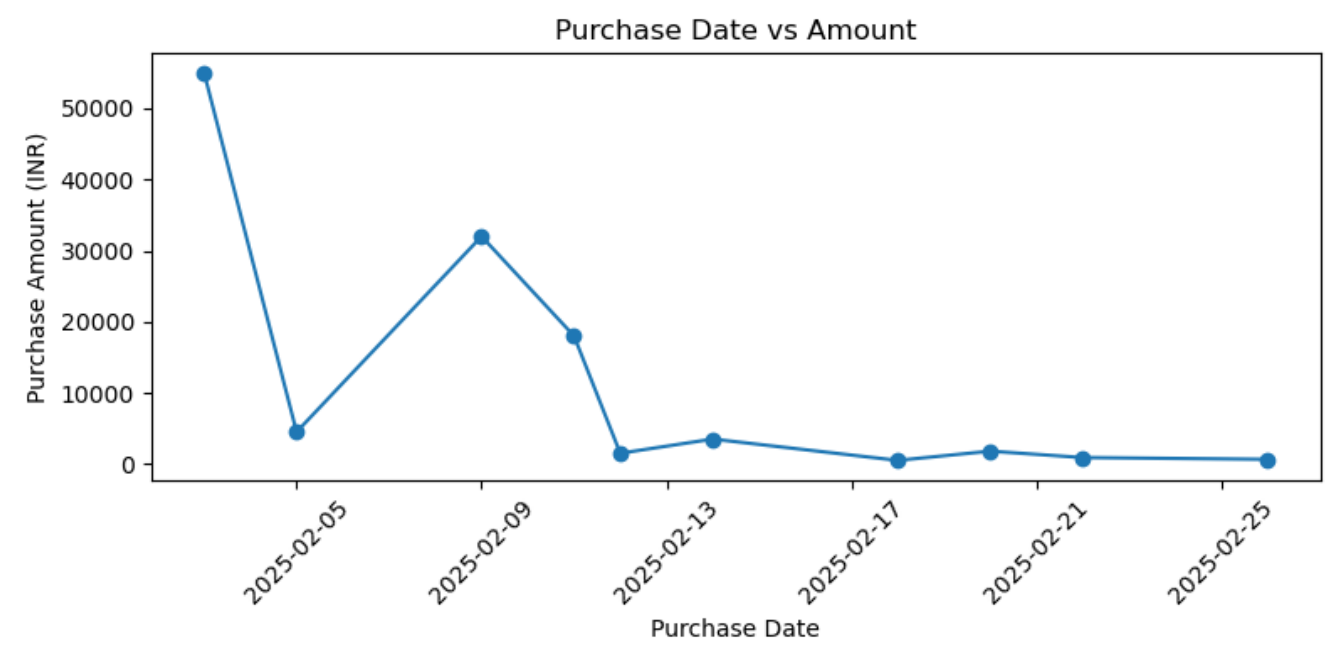


FIGURE 4



TASK 3.4 – SUMMARY

The analysis shows that Mumbai generated the highest total purchase amount among all cities, mainly due to high-value purchases made by customers in the city. The most active customers were Aarav, Bhavya, Chirag, and Eshan, with two purchases each. The average spending pattern shows that male customers had a higher average purchase amount than female customers in this dataset. The purchase data also indicates that a small number of high-value purchases contributed significantly to the overall revenue. The highest individual purchase was a laptop worth ₹55,000, which had a major impact on the total spending. Bangalore also recorded a considerable purchase amount because of the smartphone purchase worth ₹32,000. Overall, the analysis demonstrates differences in purchasing behaviour across cities, genders, and customers and shows how Pandas grouping, filtering, aggregation, and visualization can be used to understand customer purchasing patterns.